

Numerical Methods for Solving ODEs

June 4, 2026

Please visit the link for all files associated!

<https://github.com/elliottthull/Math104C-Advanced-Numerical-Analysis-Final-Project-I>

Introduction:

Ordinary Differential Equations (ODEs) arise naturally across science and engineering — from population dynamics and chemical reactions to electrical circuits and astrophysics. While many ODEs lack closed-form solutions, numerical methods allow us to approximate them to high accuracy. In this project, we investigate eight numerical solvers and analyze their behavior across several key problem settings.

This project considers the initial-value problem $y'(t) = f(t, y)$, $y(t_0) = y_0$ where we seek the function $y(t)$ on an interval $[a, b]$. Most such problems have no closed-form solution, so we approximate y at discrete points $t_i = a + ih$ using a step size h . The methods studied range from simple one-step schemes to higher-order Runge–Kutta and multistep predictor–corrector methods, and we compare their accuracy and error behavior as the step size is refined.

Numerical Methods:

We will examine methods spanning three levels of complexity. The Euler and Taylor 2nd-order methods are simple single-step schemes, with Taylor gaining accuracy as it uses derivative information to shrink that error. The Midpoint, Heun's, and RK4 methods are Runge-Kutta-type solvers that evaluate the right-hand side at intermediate points, with RK4 serving as the gold standard for single-step integration. Finally, Adams-Bashforth, Adams-Moulton, and the Predictor-Corrector are multi-step methods that achieve high accuracy with fewer function evaluations per step.

Euler's Method: The simplest one-step method. It advances the solution along the tangent line at each point:

$$w_{i+1} = w_i + h f(t_i, w_i).$$

First-order accurate, with global error $O(h)$. Easy to implement but requires very small steps for good accuracy.

Taylor Method (order n): Improves on Euler by including higher derivative terms from the Taylor expansion of y :

$$w_{i+1} = w_i + h T^{(n)}(t_i, w_i), \quad T^{(n)} = f + \frac{h}{2} f' + \dots + \frac{h^{n-1}}{n!} f^{(n-1)}.$$

Global error $O(h^n)$, but it requires computing analytic derivatives of f , which is often impractical.

Midpoint Method: A second-order Runge–Kutta method that evaluates the slope at the midpoint of the step:

$$w_{i+1} = w_i + h f\left(t_i + \frac{h}{2}, w_i + \frac{h}{2} f(t_i, w_i)\right).$$

Global error $O(h^2)$, achieving Taylor-order-2 accuracy without symbolic derivatives.

Heun's Method (Modified Euler): Another second-order Runge–Kutta method that averages the slopes at the start and predicted end of the interval:

$$w_{i+1} = w_i + \frac{h}{2}[f(t_i, w_i) + f(t_{i+1}, w_i + hf(t_i, w_i))].$$

Global error $O(h^2)$.

Runge–Kutta Method of Order 4 (RK4): The standard workhorse method, combining four slope evaluations per step:

$$w_{i+1} = w_i + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4),$$

$$k_1 = f(t_i, w_i), \quad k_2 = f(t_i + \frac{h}{2}, w_i + \frac{h}{2}k_1), \quad k_3 = f(t_i + \frac{h}{2}, w_i + \frac{h}{2}k_2), \quad k_4 = f(t_i + h, w_i + hk_3).$$

Global error $O(h^4)$, giving excellent accuracy with relatively large steps.

Adams–Bashforth Method (Explicit Multistep): Uses several previously computed points to extrapolate the next value, requiring only one new function evaluation per step. The fourth-order form is

$$w_{i+1} = w_i + \frac{h}{24}(55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3}).$$

Efficient once started (a one-step method like RK4 supplies the initial points), with global error $O(h^4)$.

Adams–Moulton Method (Implicit Multistep): Similar to Adams–Bashforth but includes the unknown point f_{i+1} , making it implicit. The third-order form is

$$w_{i+1} = w_i + \frac{h}{24}(9f_{i+1} + 19f_i - 5f_{i-1} + f_{i-2}).$$

More accurate and stable than the explicit version of the same order, but requires solving for w_{i+1} .

Predictor–Corrector Method: Combines the two multistep schemes: Adams–Bashforth *predicts* a value w_{i+1}^* , which is substituted into Adams–Moulton to *correct* it. This pairs the efficiency of the explicit predictor with the accuracy and stability of the implicit corrector, achieving $O(h^4)$ accuracy with just two function evaluations per step.

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

```
[2]: # Euler method
def euler(f, a, b, N, alpha):
    #create grid and time vector
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)

    w = np.zeros(N + 1)
    w[0] = alpha
    #loop for euler method
    for i in range(N):
        w[i + 1] = w[i] + h * f(t[i], w[i])
```

```
return t, w
```

```
[3]: # Taylor 2nd-order method
def taylor2(f, f_prime, a, b, N, alpha):
    #create grid and time vector
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha
    #loop
    for i in range(N):
        #T_2 truncation error
        T_2 = f(t[i], w[i]) + (h / 2) * f_prime(t[i], w[i])

        w[i + 1] = w[i] + h * T_2
    return t, w
```

```
[4]: # Midpoint method
def midpoint(f, a, b, N, alpha):
    #create grid and time vector
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha
    #loops for midpoint
    for i in range(N):
        w[i + 1] = w[i] + h * f(t[i] + h / 2, w[i] + (h / 2) * f(t[i], w[i]))
    return t, w
```

```
[5]: # Heun's 3rd-order method
def heuns(f, a, b, N, alpha):
    #create grid
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha
    for i in range(N):
        w[i + 1] = w[i] + (h / 4) * (
            f(t[i], w[i])
            + 3 * f(
                t[i] + (2 * h) / 3,
                w[i] + (2 * h / 3) * f(t[i] + h / 3, w[i] + (h / 3) * f(t[i], w[i]))
            )
        )
    return t, w
```

```
[6]: # Runge-Kutta 4th-order (RK4)
def RK4(f, a, b, N, alpha):
    #create grid
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha
    for i in range(N):
        #calculate 4 functions for next iteration update
        k1 = h * f(t[i], w[i])
        k2 = h * f(t[i] + h / 2, w[i] + k1 / 2)
        k3 = h * f(t[i] + h / 2, w[i] + k2 / 2)
        k4 = h * f(t[i] + h, w[i] + k3)
        w[i + 1] = w[i] + (k1 + 2 * k2 + 2 * k3 + k4) / 6
    return t, w
```

```
[7]: # Adams-Bashforth 4-step explicit method
def adams_bashforth(f, a, b, N, alpha):
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha
    # Bootstrap first 3 steps with RK4
    for i in range(3):
        k1 = h * f(t[i], w[i])
        k2 = h * f(t[i] + h / 2, w[i] + k1 / 2)
        k3 = h * f(t[i] + h / 2, w[i] + k2 / 2)
        k4 = h * f(t[i] + h, w[i] + k3)
        w[i + 1] = w[i] + (k1 + 2 * k2 + 2 * k3 + k4) / 6
    # Adams-Bashforth 4-step formula
    for i in range(3, N):
        w[i + 1] = w[i] + (h / 24) * (
            55 * f(t[i], w[i])
            - 59 * f(t[i - 1], w[i - 1])
            + 37 * f(t[i - 2], w[i - 2])
            - 9 * f(t[i - 3], w[i - 3])
        )
    return t, w
```

```
[8]: # Adams-Moulton 3-step implicit method
def adams_moulten(f, a, b, N, alpha):
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha

    # 2 steps with RK4
```

```

for i in range(2):
    k1 = h * f(t[i], w[i])
    k2 = h * f(t[i] + h / 2, w[i] + k1 / 2)
    k3 = h * f(t[i] + h / 2, w[i] + k2 / 2)
    k4 = h * f(t[i] + h, w[i] + k3)
    w[i + 1] = w[i] + (k1 + 2 * k2 + 2 * k3 + k4) / 6

for i in range(2, N):
    # Predictor (Adams-Bashforth 3-step)
    w_pred = w[i] + (h / 12) * (
        23 * f(t[i], w[i])
        - 16 * f(t[i - 1], w[i - 1])
        + 5 * f(t[i - 2], w[i - 2])
    )
    # Corrector (Adams-Moulton 3-step)
    w[i + 1] = w[i] + (h / 24) * (
        9 * f(t[i + 1], w_pred)
        + 19 * f(t[i], w[i])
        - 5 * f(t[i - 1], w[i - 1])
        + f(t[i - 2], w[i - 2])
    )
return t, w

```

```

[9]: def predictor_corrector(f, a, b, N, alpha):
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w_p = np.zeros(N + 1)
    w[0] = w_p[0] = alpha

    # First 3 steps with RK4
    for i in range(3):
        k1 = h * f(t[i], w[i])
        k2 = h * f(t[i] + h / 2, w[i] + k1 / 2)
        k3 = h * f(t[i] + h / 2, w[i] + k2 / 2)
        k4 = h * f(t[i] + h, w[i] + k3)
        w[i + 1] = w_p[i + 1] = w[i] + (k1 + 2*k2 + 2*k3 + k4) / 6

    # Adams-Bashforth 4-step predictor + Adams-Moulton 4-step corrector
    for i in range(3, N):
        # Predictor
        w_p[i + 1] = w[i] + (h / 24) * (
            55 * f(t[i], w[i])
            - 59 * f(t[i - 1], w[i - 1])
            + 37 * f(t[i - 2], w[i - 2])
            - 9 * f(t[i - 3], w[i - 3])
        )

```

```

    # Corrector
    w[i + 1] = w[i] + (h / 24) * (
        9 * f(t[i + 1], w_p[i + 1])
        + 19 * f(t[i], w[i])
        - 5 * f(t[i - 1], w[i - 1])
        + f(t[i - 2], w[i - 2])
    )

    return t, w

```

0.1 Problem A)

```

[10]: # ODE:  $y' = y - t^2 + 1$ ,  $y(0) = 1$ 
      # Exact solution:  $y(t) = (t+1)^2 - 0.5 * \exp(t)$ 

      def f_1(t, y):
          return y - t**2 + 1

      def f_1_p(t, y):
          return y - t**2 - 2*t + 1

      def y_exact(t):
          return (t + 1)**2 - 0.5 * np.exp(t)

      N_s = [10, 20, 40]

      results = {}
      for N in N_s:
          results[N] = {
              'euler': euler(f_1, 0, 2, N, alpha=0.5),
              'taylor': taylor2(f_1, f_1_p, 0, 2, N, alpha=0.5),
              'midpoint': midpoint(f_1, 0, 2, N, alpha=0.5),
              'heuns': heuns(f_1, 0, 2, N, alpha=0.5),
              'RK4': RK4(f_1, 0, 2, N, alpha=0.5),
              'adams_b': adams_bashforth(f_1, 0, 2, N, alpha=0.5),
              'adams_m': adams_moulten(f_1, 0, 2, N, alpha=0.5),
              'pred_cor': predictor_corrector(f_1, 0, 2, N, alpha=0.5),
          }

```

```

[11]: #graphing and numerical results
      method_keys = ['euler', 'taylor', 'midpoint', 'heuns', 'RK4', 'adams_b',
                     ↪ 'adams_m', 'pred_cor']
      method_labels = ['Euler', 'Taylor 2nd', 'Midpoint', "Heun's 3rd", 'RK4',
                       ↪ 'Adams-Bashforth', 'Adams-Moulton', 'Pred-Corrector']

```

```

fig, axes = plt.subplots(2, 3, figsize=(16, 14))
fig.suptitle("ODE Method Comparison:  $y' = y - t^2 + 1$ ,  $y(0) = 0.5$ ",
    ↪ fontsize=14, fontweight='bold')

colors = ['#E24B4A', '#EF9F27', '#639922', '#1D9E75', '#185FA5', '#7F77DD',
    ↪ '#D4537E', '#854F0B']

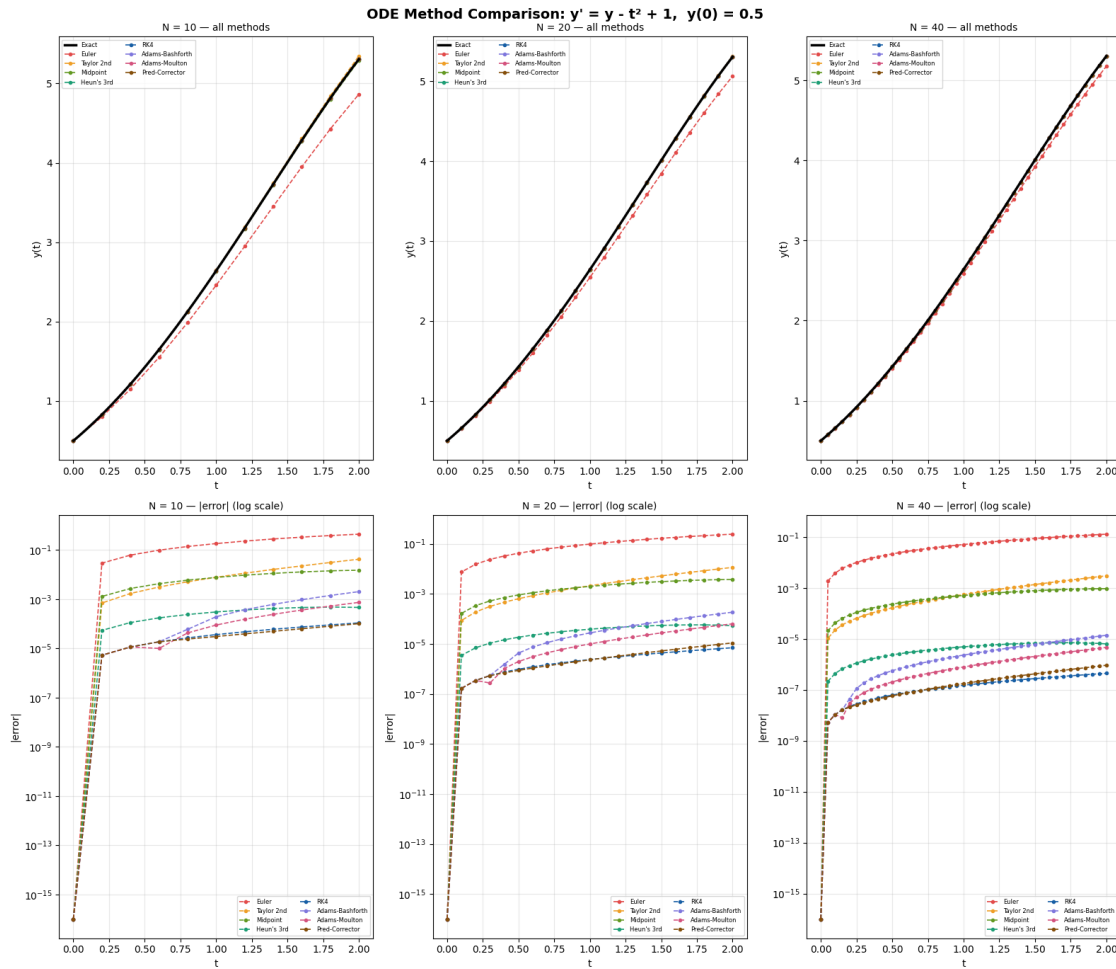
for col, N in enumerate(N_s):
    t_fine = np.linspace(0, 2, 300)

    ax_top = axes[0][col]
    ax_top.plot(t_fine, y_exact(t_fine), 'k-', linewidth=2.5, label='Exact',
    ↪ zorder=10)
    for key, label, color in zip(method_keys, method_labels, colors):
        t_m, w_m = results[N][key]
        ax_top.plot(t_m, w_m, '--o', color=color, markersize=3, linewidth=1.2,
    ↪ label=label)
    ax_top.set_title(f'N = {N} - all methods', fontsize=10)
    ax_top.set_xlabel('t')
    ax_top.set_ylabel('y(t)')
    ax_top.legend(fontsize=6, ncol=2)
    ax_top.grid(True, alpha=0.3)

    ax_mid = axes[1][col]
    for key, label, color in zip(method_keys, method_labels, colors):
        t_m, w_m = results[N][key]
        err = np.abs(w_m - y_exact(t_m))
        ax_mid.semilogy(t_m, err + 1e-16, '--o', color=color, markersize=3,
    ↪ linewidth=1.2, label=label)
    ax_mid.set_title(f'N = {N} - |error| (log scale)', fontsize=10)
    ax_mid.set_xlabel('t')
    ax_mid.set_ylabel('|error|')
    ax_mid.legend(fontsize=6, ncol=2)
    ax_mid.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



```
[12]: #create numerical tables for comparison
for N in N_s:
    t_vals = results[N]['euler'][0]    # all methods share the same t
    exact_vals = y_exact(t_vals)

    data = {'t': t_vals, 'Exact': exact_vals}
    for key, label in zip(method_keys, method_labels):
        _, w = results[N][key]
        data[label] = w

    df = pd.DataFrame(data)
    df = df.iloc[[0, N // 2, N]]    # start, half, finish only

    max_row = {'t': 'max |error|', 'Exact': ''}
    for key, label in zip(method_keys, method_labels):
        _, w = results[N][key]
        max_row[label] = np.max(np.abs(w - y_exact(t_vals)))
```



```

df = pd.concat([df, pd.DataFrame([max_row])], ignore_index=True)

print(f'\n{"="*60}')
print(f'  N = {N}  (h = {2/N:.4f})')
print(f'{"="*60}')

float_cols = [c for c in df.columns if c != 't']
pd.set_option('display.float_format', '{:.8f}'.format)
pd.set_option('display.max_columns', 20)
pd.set_option('display.width', 220)
print(df.to_string(index=False))

```

```

=====
N = 10  (h = 0.2000)
=====

```

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4
Adams-Bashforth	Adams-Moulton	Pred-Corrector				
0.00000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000
0.50000000	0.50000000	0.50000000				
1.00000000	2.64085909	2.45817600	2.64864592	2.63316675	2.64055555	2.64082269
2.64105333	2.64094815	2.64082860				
2.00000000	5.30547195	4.86578450	5.34768429	5.29036946	5.30500719	5.30536300
5.30750818	5.30621550	5.30537067				
max error		0.43968745	0.04221234	0.01510249	0.00047969	0.00010895
0.00203623	0.00074355	0.00010128				

```

=====
N = 20  (h = 0.1000)
=====

```

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4
Adams-Bashforth	Adams-Moulton	Pred-Corrector				
0.00000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000
0.50000000	0.50000000	0.50000000				
1.00000000	2.64085909	2.54375452	2.64295958	2.63887843	2.64082060	2.64085672
2.64088638	2.64086900	2.64085679				
2.00000000	5.30547195	5.06350003	5.31688258	5.30172488	5.30541871	5.30546496
5.30565651	5.30553522	5.30546102				
max error		0.24197192	0.01141063	0.00374707	0.00005734	0.00000699
0.00018456	0.00006327	0.00001093				

```

=====
N = 40  (h = 0.0500)
=====

```

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4
Adams-Bashforth	Adams-Moulton	Pred-Corrector				
0.00000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000

0.50000000	0.50000000	0.50000000				
1.00000000	2.64085909	2.59068626	2.64140447	2.64035741	2.64085425	2.64085894
2.64086138	2.64085987	2.64085891				
2.00000000	5.30547195	5.17800621	5.30843639	5.30454424	5.30546566	5.30547151
5.30548563	5.30547653	5.30547104				
max error		0.12746574	0.00296444	0.00092771	0.00000701	0.00000044
0.00001368	0.00000458	0.00000091				

```
[13]: #error tables for comparison
import pandas as pd

method_keys = ['euler', 'taylor', 'midpoint', 'heuns', 'RK4', 'adams_b', 'adams_m', 'pred_cor']
method_labels = ['Euler', 'Taylor 2nd', 'Midpoint', "Heun's 3rd", 'RK4', 'Adams-Bashforth', 'Adams-Moulton', 'Pred-Corrector']

for N in N_s:
    h = 2 / N
    t_vals = results[N]['euler'][0]
    exact_vals = y_exact(t_vals)

    data = {'t': np.round(t_vals, 4), 'Exact': exact_vals}
    for key, label in zip(method_keys, method_labels):
        _, w = results[N][key]
        data[label] = np.abs(w - exact_vals)

    df = pd.DataFrame(data)
    df = df.iloc[[0, N // 2, N]] # start, half, finish only

    # format: exact in regular float, errors in scientific notation
    fmt = {'Exact': '{:.6f}'.format}
    for label in method_labels:
        fmt[label] = '{:.2e}'.format

    print(f'\nError table | h = {h} (N = {N})\n')
    print(df.to_string(index=False, formatters=fmt))
    print()
```

Error table | h = 0.2 (N = 10)

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	0.500000	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00			
0.00e+00	0.00e+00		0.00e+00						
1.00000000	2.640859	1.83e-01	7.79e-03	7.69e-03	3.04e-04	3.64e-05			
1.94e-04	8.91e-05		3.05e-05						
2.00000000	5.305472	4.40e-01	4.22e-02	1.51e-02	4.65e-04	1.09e-04			

2.04e-03 7.44e-04 1.01e-04

Error table | h = 0.1 (N = 20)

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth
0.00000000	0.500000	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00
0.00e+00	0.00e+00		0.00e+00				
1.00000000	2.640859	9.71e-02	2.10e-03	1.98e-03	3.85e-05	2.36e-06	
2.73e-05	9.91e-06		2.29e-06				
2.00000000	5.305472	2.42e-01	1.14e-02	3.75e-03	5.32e-05	6.99e-06	
1.85e-04	6.33e-05		1.09e-05				

Error table | h = 0.05 (N = 40)

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth
0.00000000	0.500000	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00
0.00e+00	0.00e+00		0.00e+00				
1.00000000	2.640859	5.02e-02	5.45e-04	5.02e-04	4.84e-06	1.50e-07	
2.29e-06	7.84e-07		1.75e-07				
2.00000000	5.305472	1.27e-01	2.96e-03	9.28e-04	6.29e-06	4.42e-07	
1.37e-05	4.58e-06		9.13e-07				

Problem A uses the ODE $y' = y - t^2 + 1$ with exact solution $y(t) = (t + 1)^2 - \frac{1}{2}e^t$. Because the solution is smooth and non-stiff, it provides an excellent test of convergence behavior. The numerical results clearly reflect the theoretical orders of the methods. At $t=2$, Euler's error decreases from approximately $4.40 * e^{-1}$ for $N=10$ to $2.42 * e^{-1}$ for $N=20$, and then to roughly $1.27 * e^{-1}$ for $N=40$, demonstrating first-order convergence. The second-order methods (Taylor and Midpoint) show much faster improvement. Taylor's error falls from approximately $4.22 * e^{-02}$ to $2.96 * e^{-03}$, while the Midpoint method decreases from $1.51 * e^{-02}$ to roughly $9.28 * e^{-04}$. Higher-order methods perform dramatically better as expected. Heun's method, RK4, Adams-Moulton, and the Predictor-Corrector method all produce errors several orders of magnitude smaller than Euler's method. In the plots, the RK4 and multistep solutions are nearly indistinguishable from the exact solution, even with relatively large step sizes. As the step size decreases, the errors consistently decrease at rates predicted by the theoretical convergence orders, confirming the expected relationship between local truncation error and global error.

Problem B)

```
[14]: def f_2(t, y):
        #problem
        return 2*y

def y_exact_2(t):
```

```

    #exact solutio
    return np.exp(2*t)

def f_2_p(t,y):
    #derivative
    return 2 * np.exp(2*t)

#step sizes
N_s = [10, 20, 40]

results_2 = {}
for N in N_s:
    results_2[N] = {
        'euler':      euler(f_2, 0, 2, N, alpha=1),
        'taylor':     taylor2(f_2, f_2_p, 0, 2, N, alpha=1),
        'midpoint':   midpoint(f_2, 0, 2, N, alpha=1),
        'heuns':      heuns(f_2, 0, 2, N, alpha=1),
        'RK4':        RK4(f_2, 0, 2, N, alpha=1),
        'adams_b':    adams_bashforth(f_2, 0, 2, N, alpha=1),
        'adams_m':    adams_moulten(f_2, 0, 2, N, alpha=1),
        'pred_cor':   predictor_corrector(f_2, 0, 2, N, alpha=1),
    }

```

```

[15]: #plotting
fig, axes = plt.subplots(2, 3, figsize=(16, 14))

for col, N in enumerate(N_s):
    t_fine = np.linspace(0, 2, 300)
    ax_top = axes[0][col]
    ax_top.plot(t_fine, y_exact_2(t_fine), 'k-', linewidth=2.5, label='Exact',
    ↪zorder=10)
    for key, label, color in zip(method_keys, method_labels, colors):
        t_m, w_m = results_2[N][key]
        ax_top.plot(t_m, w_m, '--o', color=color, markersize=3, linewidth=1.2,
    ↪label=label)
    ax_top.set_title(f'N = {N} - all methods', fontsize=10)
    ax_top.set_xlabel('t')
    ax_top.set_ylabel('y(t)')
    ax_top.legend(fontsize=6, ncol=2)
    ax_top.grid(True, alpha=0.3)

    ax_mid = axes[1][col]
    for key, label, color in zip(method_keys, method_labels, colors):
        t_m, w_m = results_2[N][key]
        err = np.abs(w_m - y_exact_2(t_m))    # ← fixed
        ax_mid.semilogy(t_m, err + 1e-16, '--o', color=color, markersize=3,
    ↪linewidth=1.2, label=label)

```

```

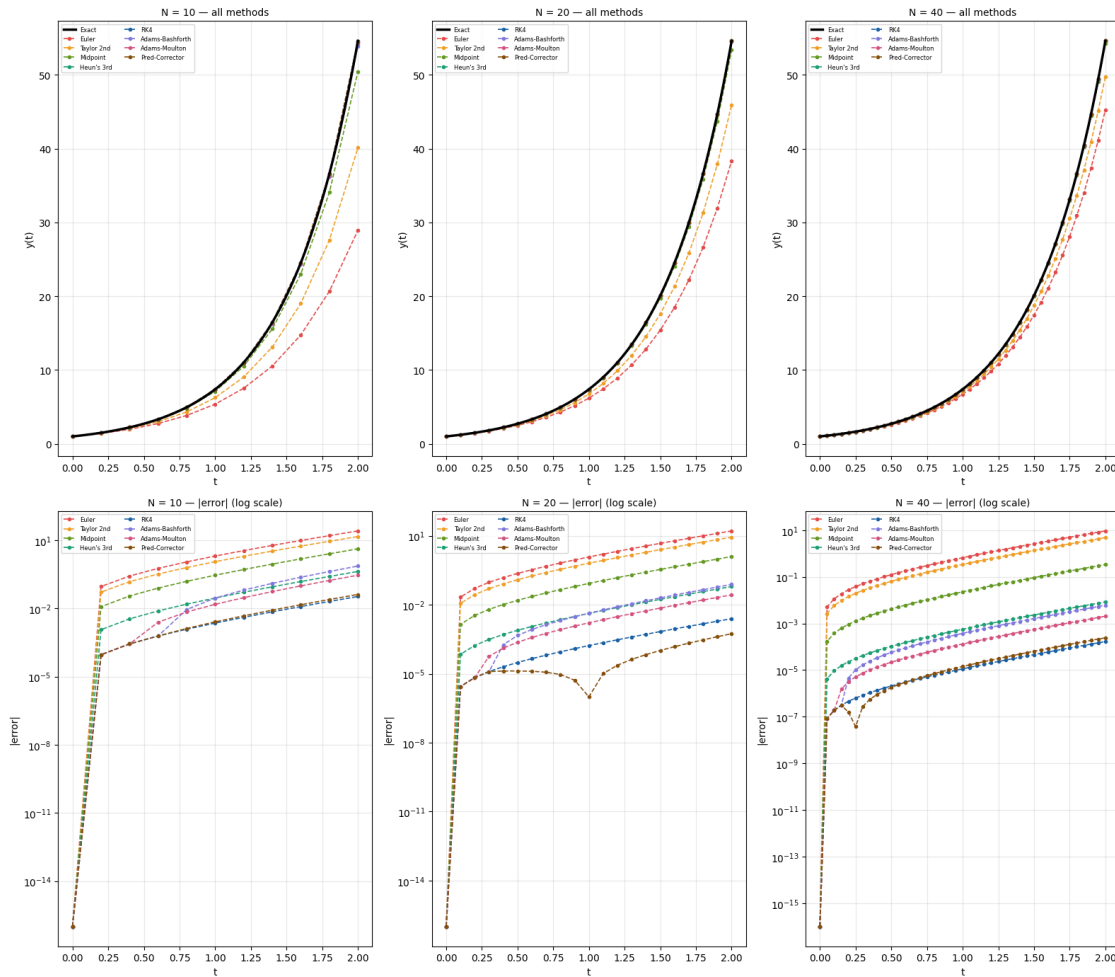
ax_mid.set_title(f'N = {N} - |error| (log scale)', fontsize=10)
ax_mid.set_xlabel('t')
ax_mid.set_ylabel('|error|')
ax_mid.legend(fontsize=6, ncol=2)
ax_mid.grid(True, alpha=0.3)

```

```

plt.tight_layout()
plt.show()

```



```

[16]: #numerical tables and solution
for N in N_s:

    t_vals = results_2[N]['euler'][0]    # all methods share the same t
    exact_vals_2 = y_exact_2(t_vals)

    data = {'t': t_vals, 'Exact': exact_vals_2}

```

```

for key, label in zip(method_keys, method_labels):
    _, w = results_2[N][key]
    data[label] = w

df = pd.DataFrame(data)
df = df.iloc[[0, N // 2, N]] # start, half, finish only

max_row = {'t': 'max |error|', 'Exact': ''}
for key, label in zip(method_keys, method_labels):
    _, w = results_2[N][key]
    max_row[label] = np.max(np.abs(w - y_exact_2(t_vals)))
df = pd.concat([df, pd.DataFrame([max_row])], ignore_index=True)

print(f'\n{"="*60}')
print(f'  N = {N}   (h = {2/N:.4f})')
print(f'{"="*60}')

float_cols = [c for c in df.columns if c != 't']
pd.set_option('display.float_format', '{:.8f}'.format)
pd.set_option('display.max_columns', 20)
pd.set_option('display.width', 220)
print(df.to_string(index=False))

```

```

=====
N = 10   (h = 0.2000)
=====

```

	t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd
RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector			
	0.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000
	1.00000000	1.00000000	1.00000000		1.00000000	
	1.00000000	7.38905610	5.37824000	6.25417693	7.10082120	7.36042176
	7.38679372	7.36051784	7.37393550		7.38649862	
	2.00000000	54.59815003	28.92546550	40.10881168	50.42166167	54.17580854
	54.56472152	53.86750332	54.30981136		54.55700735	
	max error		25.67268454	14.48933836	4.17648836	0.42234150
	0.03342851	0.73064672	0.28833867		0.04114268	

```

=====
N = 20   (h = 0.1000)
=====

```

	t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd
RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector			
	0.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000
	1.00000000	1.00000000	1.00000000		1.00000000	
	1.00000000	7.38905610	6.19173642	6.75115950	7.30463142	7.38485722
	7.38888924	7.38478191	7.38742128		7.38905708	

```

2.00000000 54.59815003 38.33759992 45.93500863 53.35764012 54.53611610
54.59568423      54.52184713      54.57125219      54.59870015
max |error|      16.26055011  8.66314141  1.24050992  0.06203394
0.00246581      0.07630290      0.02689785      0.00055011

```

```

=====
N = 40  (h = 0.0500)
=====

```

	t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd
RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector			
0.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000
1.00000000	1.00000000	1.00000000	1.00000000	1.00000000		
1.00000000	7.38905610	6.72749995	7.04734458	7.36623484	7.38848763	
7.38904477	7.38867937	7.38892422	7.38907036			
2.00000000	54.59815003	45.25925557	49.77436020	54.26141575	54.58974946	
54.59798257	54.59210455	54.59610176	54.59839432			
max error		9.33889446	4.82378983	0.33673429	0.00840057	
0.00016746	0.00604549	0.00204828	0.00024429			

```

[17]: #error tables and analysis
method_keys = ['euler', 'taylor', 'midpoint', 'heuns', 'RK4', 'adams_b', '
↪ 'adams_m', 'pred_cor']
method_labels = ['Euler', 'Taylor 2nd', 'Midpoint', "Heun's 3rd", 'RK4',
                  'Adams-Bashforth', 'Adams-Moulton', 'Pred-Corrector']

for N in N_s:
    h = 2 / N
    t_vals = results_2[N]['euler'][0]
    exact_vals_2 = y_exact_2(t_vals)

    data = {'t': np.round(t_vals, 4), 'Exact': exact_vals_2}
    for key, label in zip(method_keys, method_labels):
        _, w = results_2[N][key]
        data[label] = np.abs(w - exact_vals_2)

    df = pd.DataFrame(data)
    df = df.iloc[[0, N // 2, N]] # start, half, finish only

    # format: exact in regular float, errors in scientific notation
    fmt = {'Exact': '{:.6f}'.format}
    for label in method_labels:
        fmt[label] = '{:.2e}'.format

    print(f'\nError table | h = {h} (N = {N})\n')
    print(df.to_string(index=False, formatters=fmt))
    print()

```

Error table | h = 0.2 (N = 10)

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-
Bashforth Adams-Moulton		Pred-Corrector					
0.00000000	1.000000	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	
0.00e+00	0.00e+00	0.00e+00					
1.00000000	7.389056	2.01e+00	1.13e+00	2.88e-01	2.86e-02	2.26e-03	
2.85e-02	1.51e-02	2.56e-03					
2.00000000	54.598150	2.57e+01	1.45e+01	4.18e+00	4.22e-01	3.34e-02	
7.31e-01	2.88e-01	4.11e-02					

Error table | h = 0.1 (N = 20)

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-
Bashforth Adams-Moulton		Pred-Corrector					
0.00000000	1.000000	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	
0.00e+00	0.00e+00	0.00e+00					
1.00000000	7.389056	1.20e+00	6.38e-01	8.44e-02	4.20e-03	1.67e-04	
4.27e-03	1.63e-03	9.77e-07					
2.00000000	54.598150	1.63e+01	8.66e+00	1.24e+00	6.20e-02	2.47e-03	
7.63e-02	2.69e-02	5.50e-04					

Error table | h = 0.05 (N = 40)

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-
Bashforth Adams-Moulton		Pred-Corrector					
0.00000000	1.000000	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	
0.00e+00	0.00e+00	0.00e+00					
1.00000000	7.389056	6.62e-01	3.42e-01	2.28e-02	5.68e-04	1.13e-05	
3.77e-04	1.32e-04	1.43e-05					
2.00000000	54.598150	9.34e+00	4.82e+00	3.37e-01	8.40e-03	1.67e-04	
6.05e-03	2.05e-03	2.44e-04					

Problem B investigates the exponentially growing solution $y = e^{2t}$. This problem is particularly useful for studying error amplification because any numerical error introduced during the computation is magnified by the rapid growth of the exact solution. The results show that lower-order methods struggle significantly with this behavior. For $N=10$, Euler's maximum error reaches approximately 25.67, which is a substantial fraction of the exact solution value at $t = 2$. The second-order Taylor method reduces the maximum error to about 14.49, while the Midpoint method further lowers it to approximately 4.18. Increasing the number of steps improves the accuracy of all methods, but the higher-order methods remain substantially more accurate. For $N = 20$, RK4 achieves a maximum error of only 0.00246581, compared with 16.26055011 for Euler. At $N=40$, RK4's maximum error drops to approximately 1.67×10^{-4} , while Euler still exhibits an error greater than 9.33. Adams-Moulton and the Predictor-Corrector method also perform exceptionally well, maintaining errors on the order of 10^{-3} or smaller. The logarithmic error plots illustrate how truncation errors

are amplified over time by the exponential growth of the solution. These results demonstrate the importance of both method order and stability when solving problems in which errors naturally grow throughout the integration interval.

Problem C)

```
[18]: def f_3(t, y):
        #problem setting oscillation
        return -y**3 + np.cos(t) + np.sin(t)**3

    def f_3_p(t, y):
        #first derivative
        return -np.sin(t) + 3*np.sin(t)**2 * np.cos(t) + (-3*y**2) * (-y**3 + np.
        ↪cos(t) + np.sin(t)**3)

    def y_exact_3(t):
        #exact solution
        return np.sin(t)

    results_3 = {}
    for N in N_s:
        results_3[N] = {
            'euler': euler(f_3, 0, 2, N, alpha=0),
            'taylor': taylor2(f_3, f_3_p, 0, 2, N, alpha=0),
            'midpoint': midpoint(f_3, 0, 2, N, alpha=0),
            'heuns': heuns(f_3, 0, 2, N, alpha=0),
            'RK4': RK4(f_3, 0, 2, N, alpha=0),
            'adams_b': adams_bashforth(f_3, 0, 2, N, alpha=0),
            'adams_m': adams_moulten(f_3, 0, 2, N, alpha=0),
            'pred_cor': predictor_corrector(f_3, 0, 2, N, alpha=0),
        }

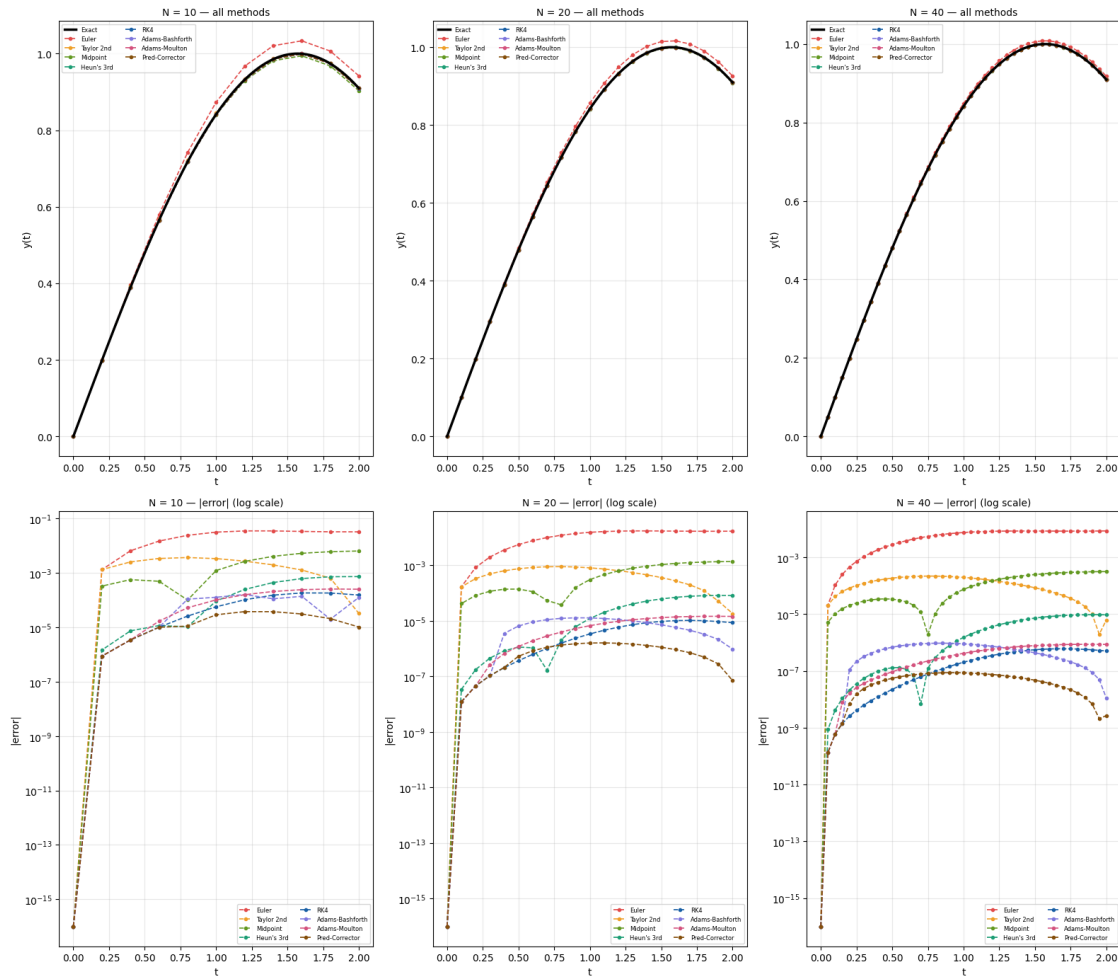
    #plotting
    fig, axes = plt.subplots(2, 3, figsize=(16, 14))
    for col, N in enumerate(N_s):
        t_fine = np.linspace(0, 2, 300)
        ax_top = axes[0][col]
        ax_top.plot(t_fine, y_exact_3(t_fine), 'k-', linewidth=2.5, label='Exact', ↪
        ↪zorder=10)
        for key, label, color in zip(method_keys, method_labels, colors):
            t_m, w_m = results_3[N][key]
            ax_top.plot(t_m, w_m, '--o', color=color, markersize=3, linewidth=1.2, ↪
            ↪label=label)
        ax_top.set_title(f'N = {N} - all methods', fontsize=10)
        ax_top.set_xlabel('t')
        ax_top.set_ylabel('y(t)')
        ax_top.legend(fontsize=6, ncol=2)
        ax_top.grid(True, alpha=0.3)
```

```

ax_mid = axes[1][col]
for key, label, color in zip(method_keys, method_labels, colors):
    t_m, w_m = results_3[N][key]
    err = np.abs(w_m - y_exact_3(t_m))
    ax_mid.semilogy(t_m, err + 1e-16, '--o', color=color, markersize=3,
↳ linewidth=1.2, label=label)
ax_mid.set_title(f'N = {N} - |error| (log scale)', fontsize=10)
ax_mid.set_xlabel('t')
ax_mid.set_ylabel('|error|')
ax_mid.legend(fontsize=6, ncol=2)
ax_mid.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



```
[19]: #numerical tables
for N in N_s:

    t_vals = results_3[N]['euler'][0]    # all methods share the same t
    exact_vals_3 = y_exact_3(t_vals)

    data = {'t': t_vals, 'Exact': exact_vals_3}
    for key, label in zip(method_keys, method_labels):
        _, w = results_3[N][key]
        data[label] = w

    df = pd.DataFrame(data)
    df = df.iloc[[0, N // 2, N]]    # start, half, finish only

    max_row = {'t': 'max |error|', 'Exact': ''}
    for key, label in zip(method_keys, method_labels):
        _, w = results_3[N][key]
        max_row[label] = np.max(np.abs(w - y_exact_3(t_vals)))
    df = pd.concat([df, pd.DataFrame([max_row])], ignore_index=True)

    print(f'\n{"="*60}')
    print(f'  N = {N}   (h = {2/N:.4f})')
    print(f'{"="*60}')

    float_cols = [c for c in df.columns if c != 't']
    pd.set_option('display.float_format', '{:.8f}'.format)
    pd.set_option('display.max_columns', 20)
    pd.set_option('display.width', 220)
    print(df.to_string(index=False))
```

```
=====
N = 10   (h = 0.2000)
=====
```

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4
Adams-Bashforth	Adams-Moulton	Pred-Corrector				
0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000
0.00000000	0.00000000	0.00000000				
1.00000000	0.84147098	0.87327943	0.84485724	0.84023981	0.84156407	0.84141374
0.84134263	0.84157379	0.84150008				
2.00000000	0.90929743	0.94188503	0.90932988	0.90284973	0.91003523	0.90914306
0.90917030	0.90955084	0.90930775				
max error		0.03526991	0.00369794	0.00644770	0.00073780	0.00018470
0.00016138	0.00025766	0.00003795				

```
=====
N = 20   (h = 0.1000)
```

```
=====
t      Exact      Euler  Taylor 2nd  Midpoint  Heun's 3rd      RK4
Adams-Bashforth Adams-Moulton Pred-Corrector
0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000
0.00000000 0.00000000 0.00000000
1.00000000 0.84147098 0.85669373 0.84227545 0.84116827 0.84148313 0.84146763
0.84145830 0.84147766 0.84147257
2.00000000 0.90929743 0.92605401 0.90927926 0.90793992 0.90937841 0.90928891
0.90929647 0.90931183 0.90929750
max |error| 0.01708925 0.00088757 0.00135751 0.00008167 0.00001030
0.00001278 0.00001453 0.00000160
=====
```

```
N = 40 (h = 0.0500)
=====
```

```
t      Exact      Euler  Taylor 2nd  Midpoint  Heun's 3rd      RK4
Adams-Bashforth Adams-Moulton Pred-Corrector
0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000
0.00000000 0.00000000 0.00000000
1.00000000 0.84147098 0.84892734 0.84166667 0.84139657 0.84147250 0.84147078
0.84147009 0.84147140 0.84147107
2.00000000 0.90929743 0.91777579 0.90929119 0.90898372 0.90930688 0.90929693
0.90929742 0.90929828 0.90929742
max |error| 0.00847836 0.00021793 0.00031370 0.00000953 0.00000060
0.00000095 0.00000086 0.00000009
=====
```

```
[20]: #error tables
method_keys = ['euler', 'taylor', 'midpoint', 'heuns', 'RK4', 'adams_b', 'adams_m', 'pred_cor']
method_labels = ['Euler', 'Taylor 2nd', 'Midpoint', "Heun's 3rd", 'RK4', 'Adams-Bashforth', 'Adams-Moulton', 'Pred-Corrector']

for N in N_s:
    h = 2 / N
    t_vals = results_3[N]['euler'][0]
    exact_vals_3 = y_exact_3(t_vals)

    data = {'t': np.round(t_vals, 4), 'Exact': exact_vals_3}
    for key, label in zip(method_keys, method_labels):
        _, w = results_3[N][key]
        data[label] = np.abs(w - exact_vals_3)

    df = pd.DataFrame(data)
    df = df.iloc[[0, N // 2, N]] # start, half, finish only

    # format: exact in regular float, errors in scientific notation
    fmt = {'Exact': '{:.6f}'.format}
```

```

for label in method_labels:
    fmt[label] = '{:.2e}'.format

print(f'\nError table | h = {h} (N = {N})\n')
print(df.to_string(index=False, formatters=fmt))
print()

```

Error table | h = 0.2 (N = 10)

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-
Bashforth	Adams-Moulton	Pred-Corrector					
0.00000000	0.000000	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	
0.00e+00	0.00e+00		0.00e+00				
1.00000000	0.841471	3.18e-02	3.39e-03	1.23e-03	9.31e-05	5.72e-05	
1.28e-04	1.03e-04		2.91e-05				
2.00000000	0.909297	3.26e-02	3.25e-05	6.45e-03	7.38e-04	1.54e-04	
1.27e-04	2.53e-04		1.03e-05				

Error table | h = 0.1 (N = 20)

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-
Bashforth	Adams-Moulton	Pred-Corrector					
0.00000000	0.000000	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	
0.00e+00	0.00e+00		0.00e+00				
1.00000000	0.841471	1.52e-02	8.04e-04	3.03e-04	1.21e-05	3.36e-06	
1.27e-05	6.68e-06		1.58e-06				
2.00000000	0.909297	1.68e-02	1.82e-05	1.36e-03	8.10e-05	8.52e-06	
9.55e-07	1.44e-05		7.02e-08				

Error table | h = 0.05 (N = 40)

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-
Bashforth	Adams-Moulton	Pred-Corrector					
0.00000000	0.000000	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	
0.00e+00	0.00e+00		0.00e+00				
1.00000000	0.841471	7.46e-03	1.96e-04	7.44e-05	1.52e-06	2.02e-07	
8.97e-07	4.17e-07		8.56e-08				
2.00000000	0.909297	8.48e-03	6.24e-06	3.14e-04	9.45e-06	5.01e-07	
1.10e-08	8.56e-07		2.66e-09				

Problem C uses the nonlinear differential equation $y' = -y^3 + \cos(t) + \sin^3(t)$ with exact solution $y(t) = \sin(t)$. This problem was selected because it contains a nonlinear term while still having a known analytical solution for comparison. The cubic damping term $-y^3$ helps suppress excessive growth, making the problem more stable than the exponential-growth problem studied in Problem

B. As expected, all methods perform better in this setting. For $N = 10$, Euler’s error near the final time remains on the order of 10^{-2} , while Taylor’s method reduces the error to approximately 10^{-3} . The higher-order methods perform significantly better, with RK4, Adams-Moulton, and the Predictor-Corrector method producing errors between 10^{-5} and 10^{-7} across most of the interval. As the step size is reduced from $h = 0.2$ to $h = 0.05$, all methods exhibit the expected decrease in error, with higher-order methods converging much more rapidly than Euler’s method. By $N = 40$, RK4 and the Predictor-Corrector method maintain errors that are extremely small and nearly indistinguishable from the exact solution on the plots. These results show that nonlinear terms do not necessarily create numerical difficulties when stabilizing effects are present. The observed behavior agrees closely with theoretical expectations regarding convergence order, error propagation, and numerical stability, with RK4 and the corrected multistep methods providing the best balance of accuracy and efficiency.

Error Analysis:

Across all three problems, the observed errors agree with theoretical predictions. Euler (order 1) converges slowest, Taylor and Midpoint (order 2) converge moderately, Heun’s (order 3) is better still, and RK4 and multi-step methods (order 4) are consistently the most accurate. Methods with the same formal order can still differ due to their leading error constants — for example, Adams-Moulton outperforms Adams-Bashforth because its implicit corrector step reduces the error constant. All methods remained stable within the step sizes tested, though Euler and Taylor would be first to lose stability if h were increased. Error propagation was well-behaved across all problems, with the nonlinear damping in Problem C actually helping to suppress error growth.

Conclusion:

Higher-order methods achieve dramatically better accuracy for the same computational cost. RK4 strikes the best balance for single-step integration, self-starting, stable, and 4th-order accurate. Multi-step methods match RK4’s accuracy with fewer evaluations per step, but require carefulness and are more sensitive to initialization errors. The Predictor-Corrector is the most sophisticated method studied, combining explicit and implicit steps to achieve near-implicit accuracy without a nonlinear solve. Correct problem setup — particularly initial conditions — proved just as important as method choice, as an incorrect initial condition rendered all methods inaccurate regardless of their order.